

4.1 MEMORY MANAGEMENT

- Memory consists of a large array of words or bytes, each with its own address.
- The CPU fetches instructions from memory according to the value of the program counter (PC).
- Main memory & registers built into the processor itself are the only storage that the CPU can access directly.
- Each process has a separate memory space.
- There is a range of legal addresses that the process may access & to the process can access only these legal addresses.
- To provide this protection, two registers are used:
 - Base Register - Holds the smallest legal physical memory address.
 - Limit Register - Specifies the size of the range.

Address Binding

- Address Binding is the process of mapping the program's logical or virtual addresses to the corresponding physical or main memory addresses.
- The binding of instructions & data to memory addresses can be done at any of the following steps:
 - ✓ Compile Time
 - ✓ Load Time
 - ✓ Execution Time

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
 - **Execution time:** If the process can be moved during its execution from one memory segment to another
 - Binding delayed until run time
 - Need hardware support for address maps (e.g., base and limit registers)

What is Process Address Space?

Address space is a space in computer memory. And process Address Space means a space that is allocated in memory for a process. Every process has an address space.

The process address space is the set of logical addresses that a process references in its code. For example, when 32-bit addressing is in use, addresses can range from 0 to 0x7fffffff; that is, 2^{31} possible numbers, for a total theoretical size of 2 gigabytes.

Address Space can be of two types

1. Physical Address Space
2. Virtual Address Space

Physical address space

The physical address is the actual location in the memory that exist physically. System accesses the data in the main memory, with the help of physical address. Everything in the computer has a unique physical address. We need to mapped it to make the address accessible. MMU is responsible for mapping.

Virtual Address Space

Virtual Address Space is an address space that is created outside the main memory inside the virtual memory, and it is created in the hard disk.

When our main memory is less and we want to get more benefit from this less memory, then we create virtual memory

Logical Address is the address is always generated by CPU while running a program. The logical address is also called virtual address.

Virtual address does not exist physically, so we can say that physical address is the logical address.

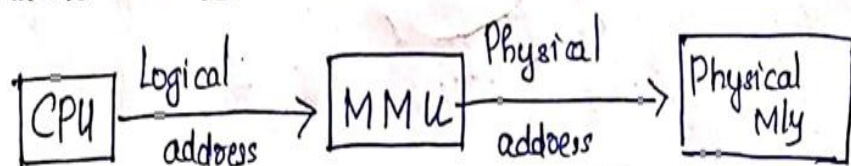
The set of all logical addresses generated by a program is referred to as a **logical address space**. The set of all physical addresses corresponding to these logical addresses is referred to as a **physical address space**.

The runtime mapping from virtual to physical address is done by the memory management unit (MMU) which is a hardware device. MMU uses following mechanism to convert virtual address to physical address.

- The value in the base register is added to every address generated by a user process, which is treated as offset at the time it is sent to memory. For example, if the base register value is 10000, then an attempt by the user to use address location 100 will be dynamically reallocated to location 10100.
- The user program deals with virtual addresses; it never sees the real physical addresses.

PARAMETER	LOGICAL ADDRESS	PHYSICAL ADDRESS
Basic	ADDRESS generated by CPU is called LOGICAL ADDRESS.	PHYSICAL ADDRESS is the actual location in a memory unit.
Generation	generated by the CPU	Computed by MMU
Access	logical address can be used to access the physical address by the user.	The user can't directly access physical address but it is possible indirectly.
Visibility	logical address of a program is visible to the user.	physical address of program is not visible to the user.
Address Space	set of all logical addresses generated by CPU are called logical addresses.	Addresses mapped to the corresponding logical addresses.

- * Logical & Physical address are the same in compile time & load time address-binding scheme.
- * Logical & physical address are different in execution time address binding scheme.



Memory Management Unit

- It is a hardware device that maps virtual addresses to physical address.
- In MMU scheme, the value in relocation reg is added to every address generated by the user process at the time it is sent to memory.
- The user program ~~always~~ deals with logical address. It never sees the real physical address.

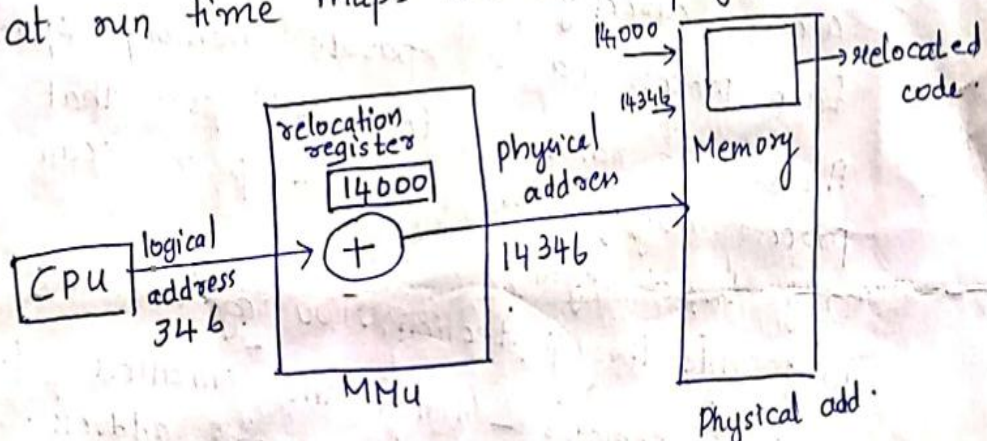
- Logical address (Virtual address) - Generated by the CPU. Also referred to as virtual address.

- Physical address - Address seen by the memory unit.

- Logical address space is the set of all logical addresses generated by a pgm.

- Physical address space is the set of all physical addresses generated by a pgm.

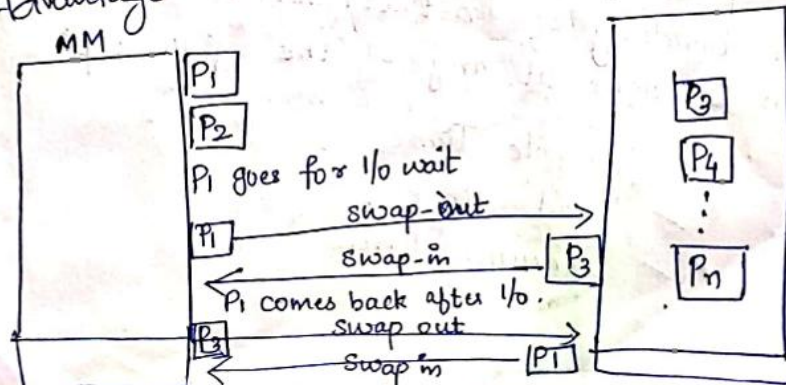
- Memory Management Unit (MMU) - H/w devices that at run time maps virtual to physical address.



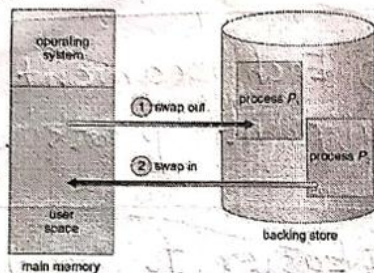
4.2 Swapping

- Swapping used in order to implement multi-programming.
- In multiprogramming, can store multiple process in main mly. So that CPU can execute all process in an efficient manner.

Advantage - utilize CPU performance in efficient manner



- A process must be in memory to be executed.
- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution.
- Swapping variant used for priority-based scheduling algorithms are called roll out, roll in- lower-priority process is swapped out so higher-priority process can be loaded and executed.
- Swapping requires a backing store.
- The backing store is commonly a fast disk. It must be large enough to accommodate copies of all memory images for all users, and it must provide direct access to these memory images.
- The system maintains a ready queue consisting of all processes whose memory images are on the backing store or in memory and are ready to run.
- Whenever the CPU scheduler decides to execute a process, it calls the dispatcher.
- The dispatcher checks to see whether the next process in the queue is in memory. If it is not; and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process. It then reloads registers and transfers control to the selected process.



Swapping of two processes using a disk as a backing store.

MEMORY MANAGEMENT TECHNIQUES

Memory Management Techniques are basic techniques that are used in managing the memory in operating system. Memory Management Techniques are basically classified into two categories:

- (i) Contiguous
- (ii) Non-contiguous

Contiguous Memory Management Techniques:

In this technique, memory is allotted in a continuous way to the processes.

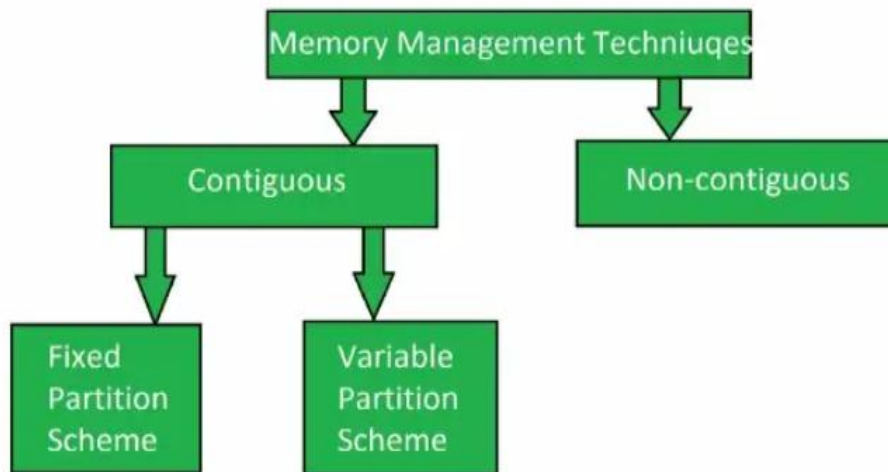
In the procedure of contiguous memory allocation, all the accessible memory space stays together in one spot. It implies openly accessible memory partitions are not dissipated to a whole extent over the entire memory space.

Non-Contiguous memory allocation

Non-Contiguous memory allocation is basically a method on the contrary to contiguous allocation method, allocates the memory space present in different locations to the process as per its requirements.

<u>Contiguous v/s Non-Contiguous Allocation</u>	
<u>Way of organizing program in m/y</u>	
<u>Contiguous</u>	<u>Non-Contiguous</u>
① Program must exist as a single block of contiguous address.	① Programs divided into chunks called segment.
② Sometimes it is impossible to find large enough block.	② Each segment can be placed in different parts of memory.
③ Low Overhead.	③ Easier to find "holes" (free space) in which segment it will fit

It has two types:



Fixed Partition Scheme:

In the fixed partition scheme, memory is divided into fixed number of partitions. Fixed means numbers of partitions are fixed in the memory. In the fixed partition, in every partition only one process will be accommodated. Degree of multi-programming is

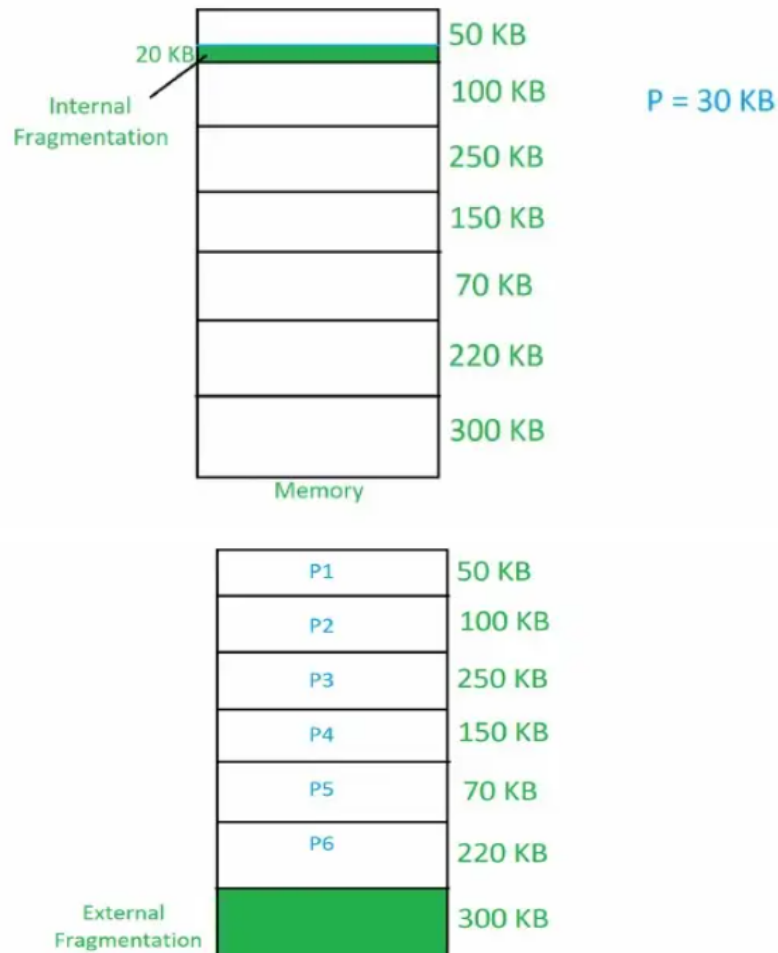
Internal Fragmentation is found in fixed partition scheme. To overcome the problem of internal fragmentation, instead of fixed partition scheme, variable partition scheme is used.

Variable Partition Scheme:

In the variable partition scheme, initially memory will be single continuous free block. Whenever the request by the process arrives, accordingly partition will be made in the memory. If the smaller processes keep on coming then the larger partitions will be made into smaller partitions.

restricted by number of partitions in the memory. Maximum size of the process is restricted by maximum size of the partition. Every partition is associated with the *limit registers*.

- **Limit Registers:** It has two limit:
 - **Lower Limit:** Starting address of the partition.
 - **Upper Limit:** Ending address of the partition.



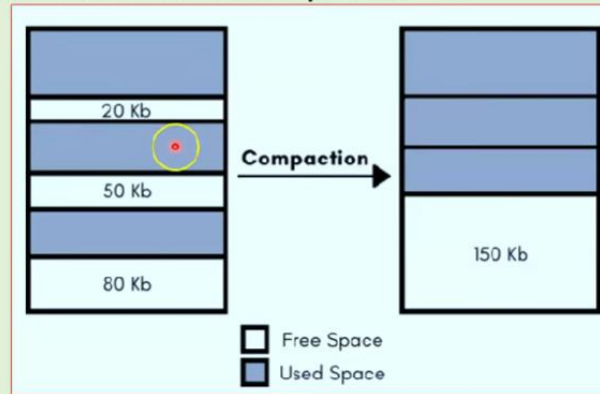
External Fragmentation is found in variable partition scheme. To overcome the problem of external fragmentation, compaction technique is used or non-contiguous memory management techniques are used.

Compaction:

Moving all the processes toward the top or towards the bottom to make free available memory in a single continuous place is called as compaction. Compaction is undesirable to implement because it interrupts all the running processes in the memory.

➤ Memory compaction

- External fragmentation **can be reduced** using memory compaction. In memory compaction, we move the used space in upper memory or in lower memory. That is used space is moved in one place and unused are collected in another place.



* Fixed Partitioning

- Main mly is divided into no. of static partitions at system generation time.
 - * Equal size partition
 - * Unequal size partition

Pros.

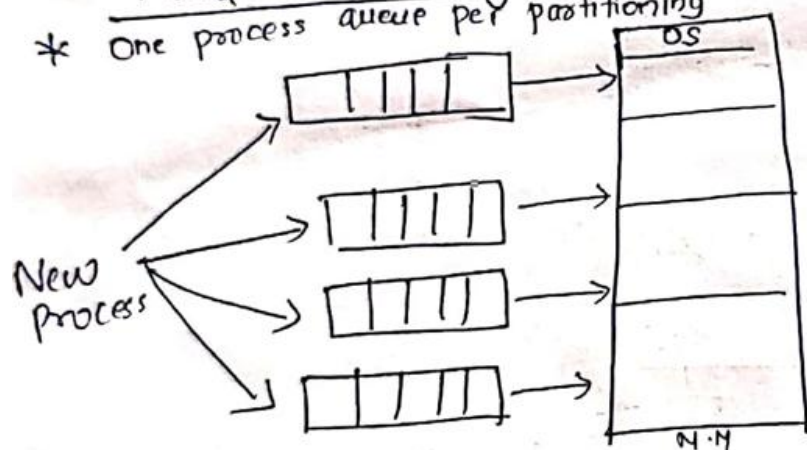
- Simple to implement.
- Little OS overhead.

Cons.

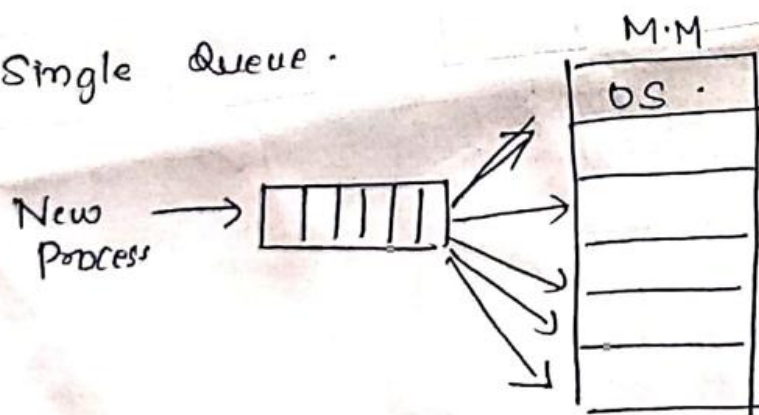
- Inefficient
- Maximum no. of active process - fixed

Fixed Partitioning Placement Algorithm

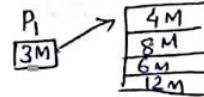
* One process queue per partitioning



* Single Queue .



* Dynamic Partitioning



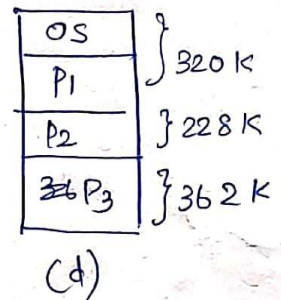
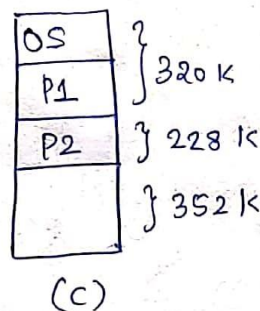
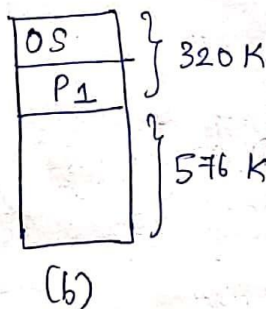
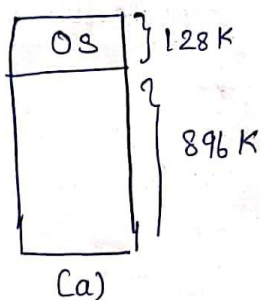
- Partitions are created dynamically.
- Each process is loaded into a partition of exactly the same size of that process.

- External fragmentation exists when there is enough total memory space to satisfy a request but the available spaces are not contiguous; storage is fragmented into a large number of small holes.
- In the worst case, it could have a block of free (or wasted) memory between every two processes. If all these small pieces of memory were in one big free block instead, it might be able to run several more processes.
- Memory fragmentation can be internal as well as external.
- The general approach to avoiding this problem is to break the physical memory into fixed-sized blocks and allocate memory in units based on block size.
- The memory allocated to a process may be slightly larger than the requested memory. The difference between these two numbers is internal fragmentation; memory that is internal to a partition.
- Compaction is the solution for external fragmentation- shuffles the memory contents so as to place all free memory together in one large block.

Placement Algorithm

- First fit : First hole that is big enough is allocated to program.
- Best fit : Smallest hole that is big enough is allocated to pgm.
- Worst fit : Largest hole that is big enough is allocated to pgm.

Eg:



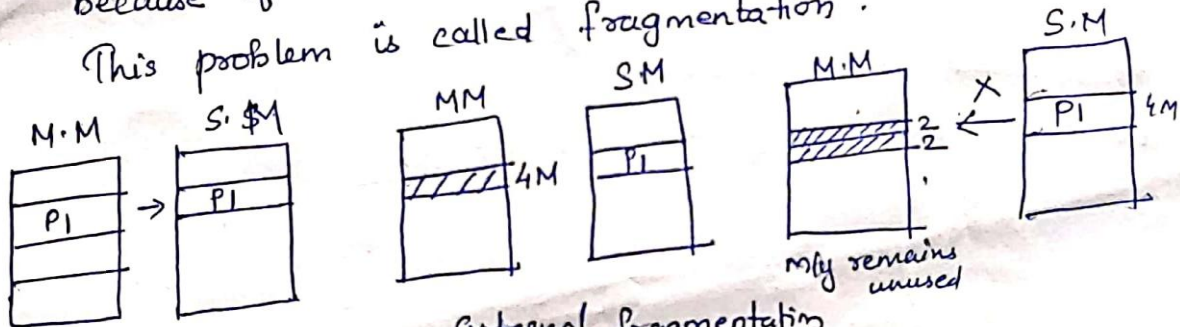
Memory Allocation

- A simplest method for allocating memory is to divide memory into several fixed-sized partitions.
Each partition may contain exactly one process. Thus, the degree of multiprogramming is bound by the number of partitions. In this multiple partitioned method, when a partition is free; a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.
- In the variable partition scheme, the operating system keeps a table indicating which parts of memory are available and which are occupied. Initially, all memory is available for user processes and is considered one large block of available memory, a hole. Eventually, memory contains a set of holes of various sizes.
- As processes enter the system, they are put into an input queue. The operating system takes into account the memory requirements of each process and the amount of available memory space in determining which processes are allocated memory.
- When a process is allocated space, it is loaded into memory, and it can then compete for CPU time. When a process terminates, it releases its memory which the operating system may then fill with another process from the input queue.
- In general, the memory blocks available comprise a set of holes of various sizes scattered throughout memory. When a process arrives and needs memory, the system searches the set for a hole that is large enough for this process. If the hole is too large, it is split into two parts. One part is allocated to the arriving process; the other is returned to the set of holes.
- When a process terminates, it releases its block of memory, which is then placed back in the set of holes.
- If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole. At this point, the system may need to check whether there are processes waiting for memory and whether this newly freed and recombined memory could satisfy the demands of any of these waiting processes.
- **Dynamic storage allocation problem** concerns how to satisfy a request of size n from a list of free holes. Solutions for this problems are
 - **First fit.** Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.
 - **Best fit.** Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.
 - **Worst fit.** Allocate the largest hole. Search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach
- Both the first-fit and best-fit strategies for memory allocation suffer from external fragmentation.
- As processes are loaded and removed from memory, the free memory space is broken into little pieces.

FRAGMENTATION

- A process is loaded & removed from memory, the free memory space is broken into little pieces.
- After sometimes, that process cannot be allocated to memory because of small size & memory block remains unused.

This problem is called fragmentation.

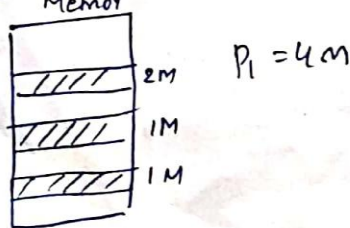


Fragmentation

- External fragmentation
- Internal fragmentation

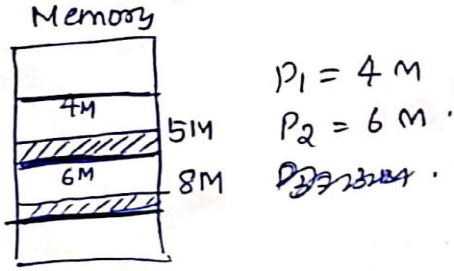
External fragmentation

- Total memory space is enough to satisfy a request or reside a process in it, but it is not contiguous, so it cannot be used.



Internal fragmentation

- Memory block assigned to process is bigger. Some portion of memory is left unused, as it cannot be used by another process.



MEMORY API

- Memory API , including the stack and heap memory, the malloc() and free() functions, and common memory management mistakes.
- Key concepts include memory allocation, memory deallocation, and the differences between stack and heap memory

- In running a C program, two types of memory are allocated.
- The first is called **stack memory**, allocations and de-allocations of it are managed implicitly by the **compiler** for the programmer, for this reason, it is sometimes called automatic memory.
- Declaring memory on the stack in C is easy. For example, you need space in a function **func()** for an x integer. To declare such a piece of memory, the snippet is :

```
void func()
{
    int x; // declares an integer on the stack
    ...
}
```

The compiler does the rest, making sure to make space on the stack when you call into **func()**.

4

Stack memory

- Variables declared inside a function are stored in the stack memory
- Stack memory is used to store local variables and function call
- Stack memory is always referenced in LIFO order
- Stack memory is very fast when compared to heap memory
- Stack memory has a limited size

The compiler does the rest, making sure to make space on the stack when you call into **func()**.

Declaring a variable in a function:

```
void func()
{
    int a = 10; // declares a variable in
                the stack memory
}
```

- Second type of memory, called **heap memory**, where all allocations and de-allocations are explicitly handled by **the programmer**.
- Here is an example of how one might allocate an integer on the heap:

```
void func()
{
    int *x = (int *) malloc(sizeof(int));
    ...
}
```


Heap memory

- Variables declared in the heap memory are accessible globally
- Heap memory is not as fast as stack memory
- Heap memory has no size limit
- Heap memory is allocated during runtime

Declaring a variable in the heap memory:

- `int *a = (int *)malloc(sizeof(int));` // declares a variable in the heap memory

When `sizeof()` is used with the data types such as `int`, `float`, `char`... etc. it simply returns the amount of memory allocated to that data type.

```
#include <stdio.h>
int main()
{
    printf("%lu\n",
sizeof(char));
    printf("%lu\n", sizeof(int));
    printf("%lu\n", sizeof(float));
    printf("%lu", sizeof(double));
    return 0;
}
```

Output:1 4 4 8

The free() Function

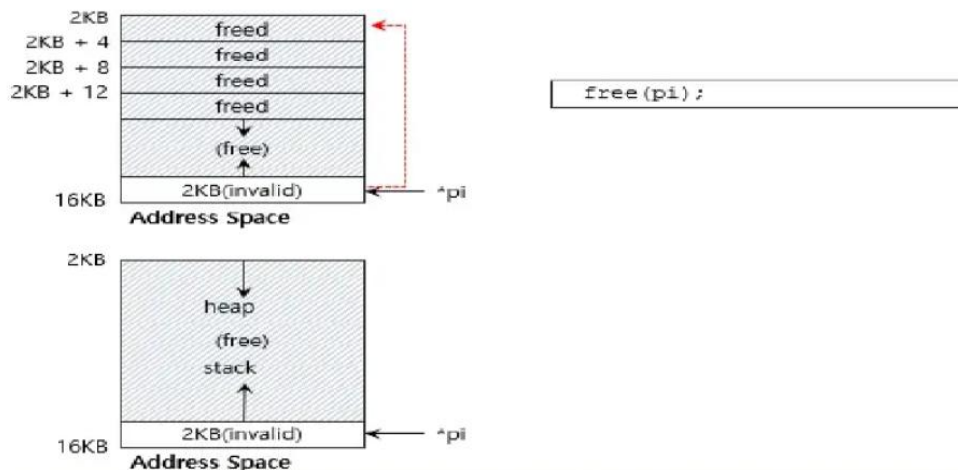
The `free()` function is used to deallocate memory allocated by the `malloc()` function.

The `free()` function takes a pointer to the memory to be deallocated as an argument.

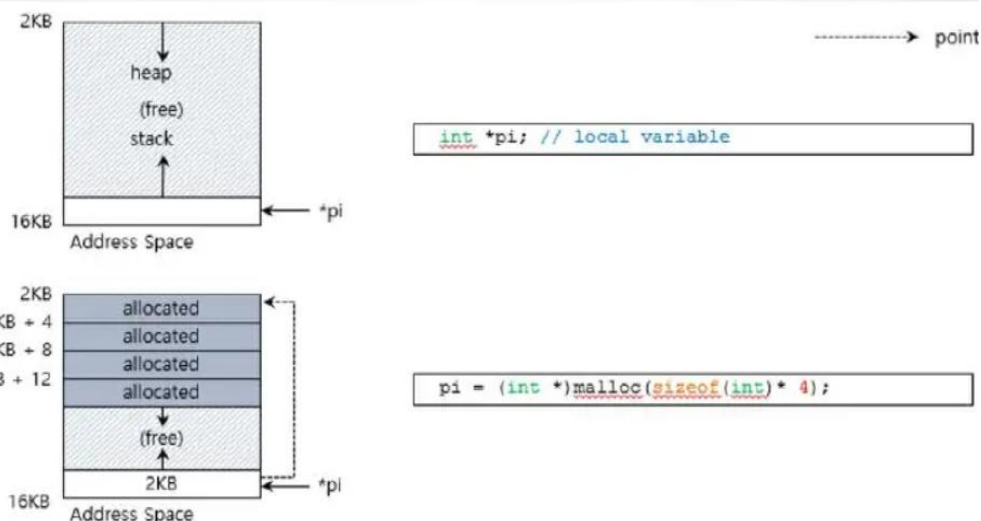
The function does not return anything.

```
int *a = (int *)malloc(sizeof(int));
free(a); // deallocates the memory allocated by malloc()
```

Memory Freeing



MEMORY ALLOCATING



Common Memory Management Errors

1. Forgetting to Allocate Memory

Some functions expect memory to be allocated before use.

Example Error:

```
char *dst;
```

```
strcpy(dst, "hello"); // Error
```

- dst points to unallocated memory
- Causes segmentation fault

Correct Approach:

```
char *dst = (char *)malloc(strlen("hello") + 1);  
strcpy(dst, "hello");
```

✦ Always allocate memory before using it.

2. Not Allocating Enough Memory (Buffer Overflow)

Allocating less memory than required causes buffer overflow.

Example:

```
char *dst = (char *)malloc(strlen(src)); // Insufficient memory
```

- Missing space for the null character `\0`
- May overwrite nearby memory
- Leads to crashes or security vulnerabilities

✦ Always allocate `strlen(string) + 1` bytes for strings.

3. Forgetting to Initialize Allocated Memory

- `malloc()` does not initialize memory
- Reading uninitialized memory leads to unpredictable behavior

Problem:

- Program reads garbage values
- Results may be random or harmful

✦ Initialize memory after allocation or use `calloc()`.

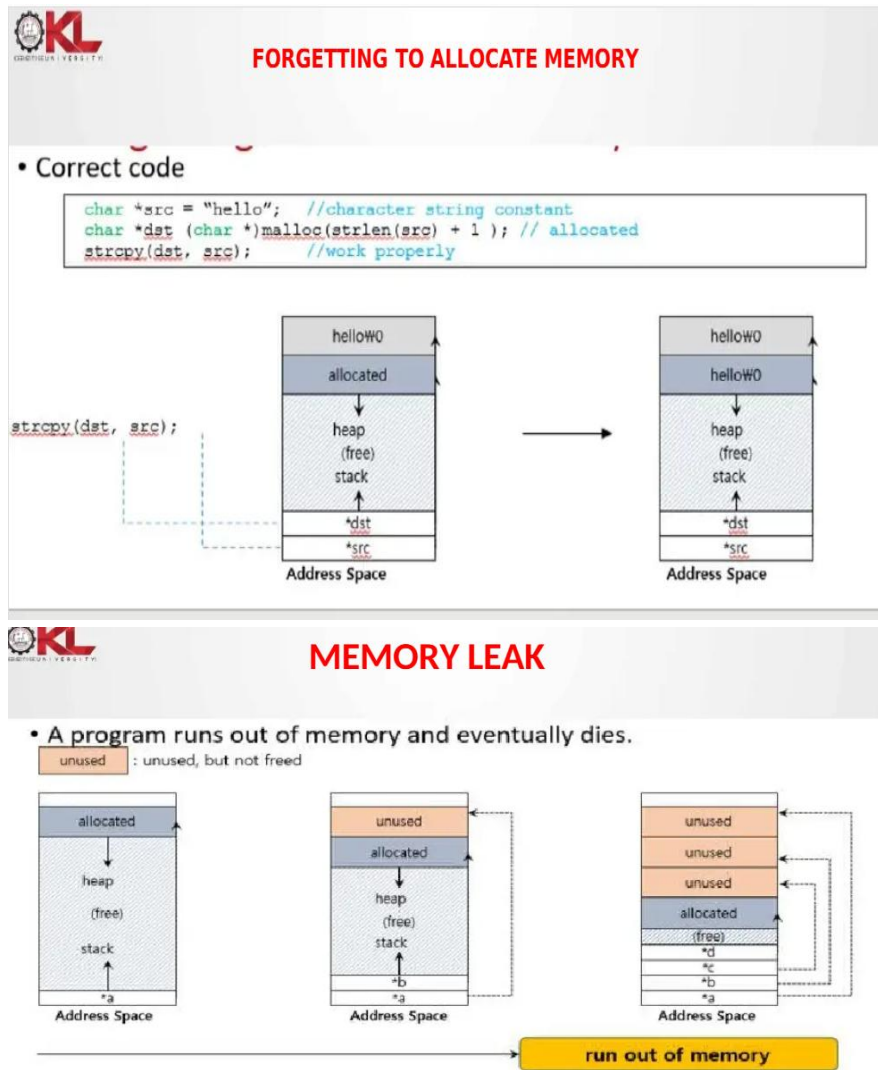
4. Forgetting to Free Memory (Memory Leak)

- Occurs when allocated memory is never freed
- Severe problem in long-running programs (servers, OS)

Effects:

- Memory usage keeps increasing

- System eventually runs out of memory
-  Always free memory when it is no longer needed.



5. Freeing Memory Before Use Is Complete (Dangling Pointer)

- Memory is freed while still in use
- Pointer continues to reference freed memory

Result:

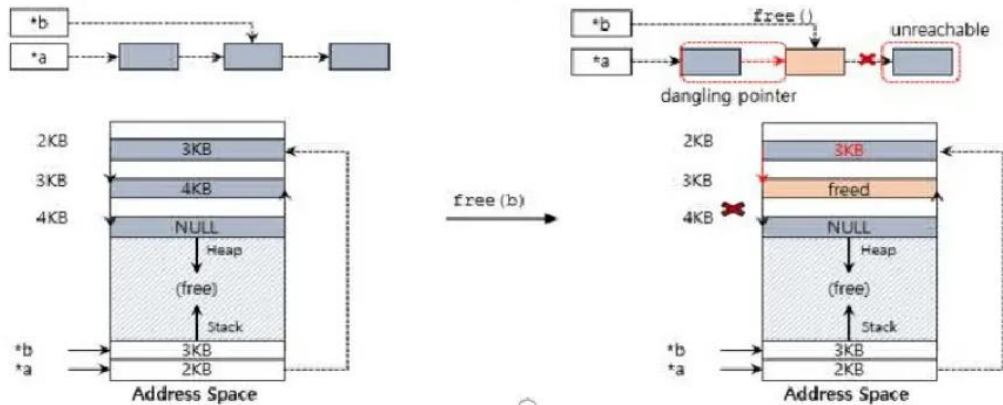
- Program crash
- Data corruption

 Never access memory after calling free().

DANGLING POINTER

1. Freeing memory before it is finished using

- A program accesses to memory with an invalid pointer



16

6. Freeing Memory More Than Once (Double Free)

- Same memory is freed multiple times

Effects:

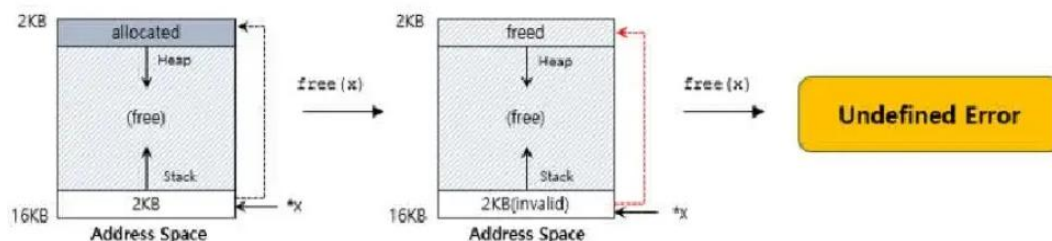
- Undefined behavior
- Program crashes or allocator corruption

✦ Each allocated block should be freed exactly once.

DOUBLE FREE

- Free memory that was freed already.

```
int *x = (int *)malloc(sizeof(int)); // allocated
free(x); // free memory
free(x); // free repeatedly
```



Undefined Error

7. Calling free() Incorrectly

- free() must be passed only pointers returned by malloc()
- Passing invalid or modified pointers causes errors

✦ Never free memory not allocated dynamically.



OTHER MEMORY APIS: CALLOC()

```
#include <stdlib.h>

void *calloc(size_t num, size_t size)
```

- calloc() allocates the memory and also initializes the allocated memory block to zero. If we try to access the content of these blocks then we'll get 0 whereas if u want to access the content of the block which was allocated by malloc(), it will return garbage values.
- Allocate memory on the heap and zeroes it before returning.
 - Argument
 - size_t num : number of blocks to allocate
 - size_t size : size of each block(in bytes)
 - Return
 - Success : a void type pointer to the memory block allocated by calloc
 - Fail : a null pointer



MEMORY APIS: REALLOC()

```
#include <stdlib.h>

void *realloc(void *ptr, size_t size)
```

- The realloc() function is used to resize allocated memory without losing old data.
- Change the size of memory block.
 - A pointer returned by realloc may be either the same as ptr or a new.
 - Argument
 - void *ptr: Pointer to memory block allocated with malloc, calloc or realloc
 - size_t size: New size for the memory block(in bytes)
 - Return
 - Success: Void type pointer to the memory block
 - Fail : Null pointer

Memory Virtualizing – Address Translation

- ▣ Efficiency and control are the two main goals of the operating systems.
- ▣ In memory virtualizing, efficiency and control are attained by hardware support.
 - ◆ Efficiency: fast address translation
 - ◆ Control: no process can access any memory but its own -- protection
- ▣ **Hardware-based address translation**
 - ◆ Turning virtual addresses (the process's view of memory) provided by the instruction to physical addresses (the actual physical memory view) where the information is actually stored in the memory.
 - ◆ Address translation is **transparent** to the process.
- ▣ **Base-and-bounds (dynamic relocation) virtualization**
 - ◆ Base register: used to transform virtual addresses into physical addresses
 - ◆ Bound register: check the address generated by the process is in the bounds – application protection.

Example: Address Translation

- ▣ C - Language code

```
void func() {  
    int x = 3000;  
    x = x + 3; // this is the line of code we are interested in  
    ...  
}
```

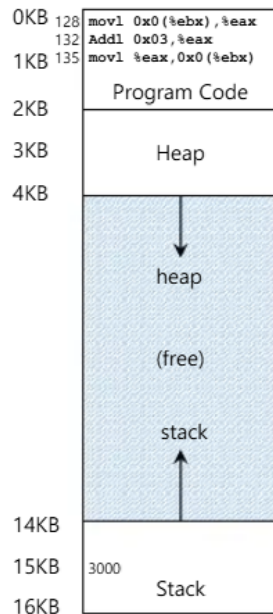
- ◆ **Load** a value from memory
- ◆ **Increment** it by three
- ◆ **Store** the value back into memory

Example: Address Translation(Cont.)

▣ Assembly

```
128 : movl 0x0(%ebx), %eax    ; load 0+ebx into eax
132 : addl $0x03, %eax        ; add 3 to eax register
135 : movl %eax, 0x0(%ebx)    ; store eax back to mem
```

- ◆ Presume that the address of '**x**' has been place in **ebx** register.
- ◆ **Load** the value at that address into **eax** register.
- ◆ **Add** 3 to **eax** register.
- ◆ **Store** the value in **eax** back into memory.

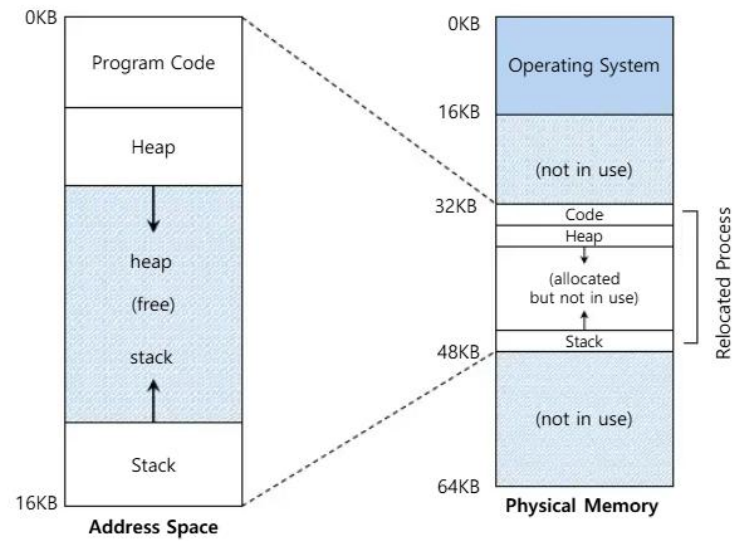


Assume: ebx = 15KB

- Fetch instruction at address 128
- Execute this instruction (load from address 15KB)
- Fetch instruction at address 132
- Execute this instruction (no memory reference)
- Fetch the instruction at address 135
- Execute this instruction (store to address 15 KB)

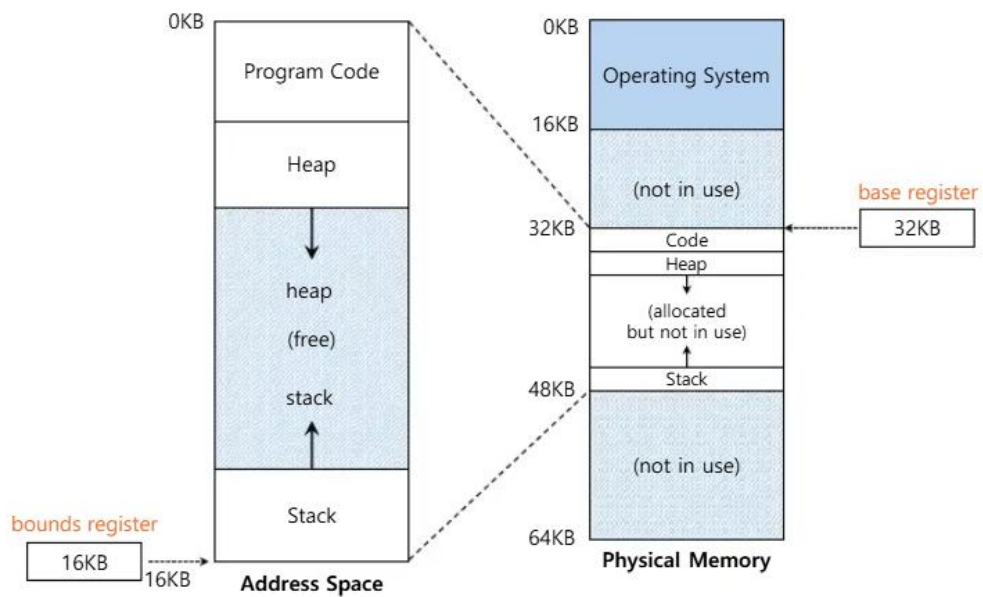
Relocation Address Space

- The OS places the process **somewhere else** in physical memory, not at address 0.
 - ♦ The address space starts at address 0.



6

Base and Bounds Register



Dynamic (Hardware base) Relocation

- When a program starts running, the OS decides **where** in physical memory a process should be **loaded**.
 - ♦ Set the **base** register a value.
 - ♦ Address Translation: Transform a virtual address into a physical address at **runtime**, also referred to as **dynamic** relocation.

$$\text{physical address} = \text{virtual address} + \text{base}$$

- ♦ Every virtual address must **not be greater than bound** and **negative**.

$$0 \leq \text{virtual address} < \text{bounds}$$

8

Relocation and Address Translation

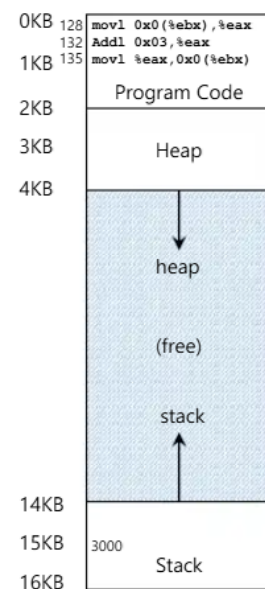
128 : `movl 0x0(%ebx), %eax`

- ♦ **Fetch** instruction at virtual address 128 (**PC** = 128)

$$32896 = 128 + 32\text{ KB}(\text{base})$$

- ♦ **Execute** this instruction
 - Load from virtual address 15KB (**ebx** = 15 KB)

$$47\text{KB} = 15\text{KB} + 32\text{KB}(\text{base})$$



1. Explain Address Translation using Base and Bounds technique.

Address translation is a mechanism that allows a process to use virtual addresses while the hardware converts them into physical addresses. In the base and bounds technique, each process is assigned a contiguous region of physical memory.

The **base register** holds the starting physical address of the process, and the **bounds register** holds the size of its address space. When a process generates a virtual address, the MMU first checks whether the address is less than the bounds value. If it is valid, the base value is added to the virtual address to obtain the physical address. If the address exceeds the bounds, an exception is generated.

This technique provides memory protection, isolation between processes, and efficient execution with minimal overhead.

2. Describe the role of the Operating System in address translation.

The operating system plays a crucial role in address translation. It allocates memory for processes and loads them into physical memory. The OS sets the base and bounds registers before a process starts executing. During context switches, it saves and restores these registers to ensure correct address translation.

The OS also maintains free memory lists, reclaims memory when processes terminate, and handles exceptions caused by illegal memory accesses. By coordinating with hardware, the OS ensures protection, isolation, and efficient memory utilization.

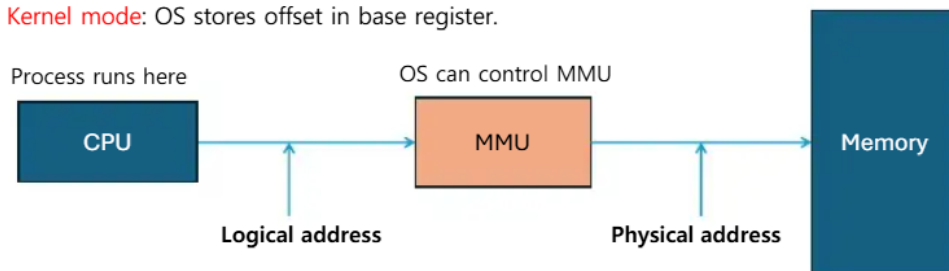
3. Compare Static Relocation and Dynamic Relocation.

Feature	Static Relocation	Dynamic Relocation
Time of relocation	Before execution	During execution
Hardware support	Not required	Required (MMU)
Protection	Not provided	Provided
Flexibility	Low	High
Modern usage	Rare	Common

Dynamic relocation is preferred because it provides protection and allows processes to be moved easily in memory.

Memory Management Unit (MMU)

- ❑ Memory Management Unit (MMU)
 - ♦ Part of the processor that helps with address translation
 - ♦ Address translation, protection control, exceptions generation and handling
- ❑ MMU dynamically changes process address at every memory reference (**User mode**)
 - ♦ Process generates **logical** or **virtual** addresses (in their address space)
 - ♦ Memory hardware uses **physical** or **real** addresses
- ❑ Each process has different value in base register
 - ♦ Dynamic relocation by changing value of base register!
 - ♦ **Kernel mode**: OS stores offset in base register.



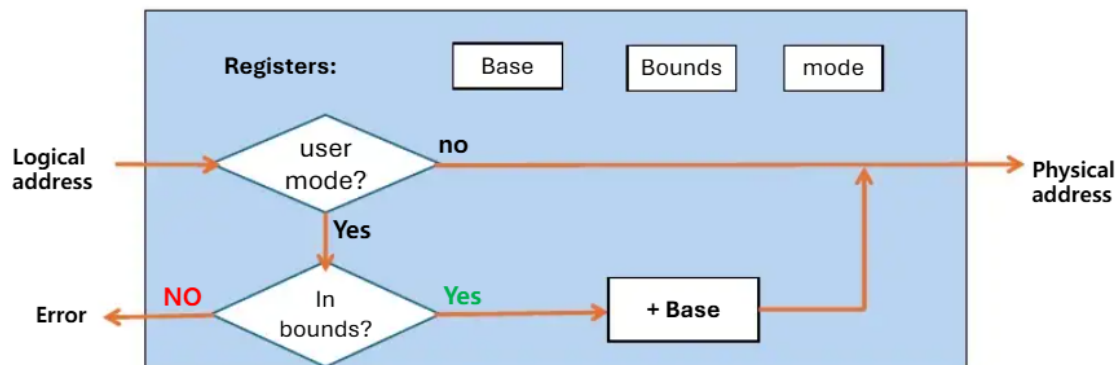
12

Dynamic Relocation: Hardware Requirements

Hardware Requirements	Notes
Privileged mode	<i>Needed to prevent user-mode processes from executing privileged operations</i>
Base/bounds registers	<i>Need pair of registers per CPU to support address translation and bounds checks</i>
Ability to translate virtual addresses and check if within bounds	<i>Circuitry to do translations and check limits; in this case, quite simple</i>
Privileged instruction(s) to update base/bounds	<i>OS must be able to set these values before letting a user program run</i>
Privileged instruction(s) to register exception handlers	<i>OS must be able to tell hardware what code to run if exception occurs</i>
Ability to raise exceptions	<i>When processes try to access privileged instructions or out-of-bounds memory</i>

Implementation of Base-and-Bounds

- Translation on every memory access of user process
 - ♦ MMU adds **base register** to logical address to form physical address
- Bounds check
 - ♦ MMU compares logical address to bounds register
 - if logical address is greater, then generate error



14

Dynamic Relocation: Operating System Responsibility

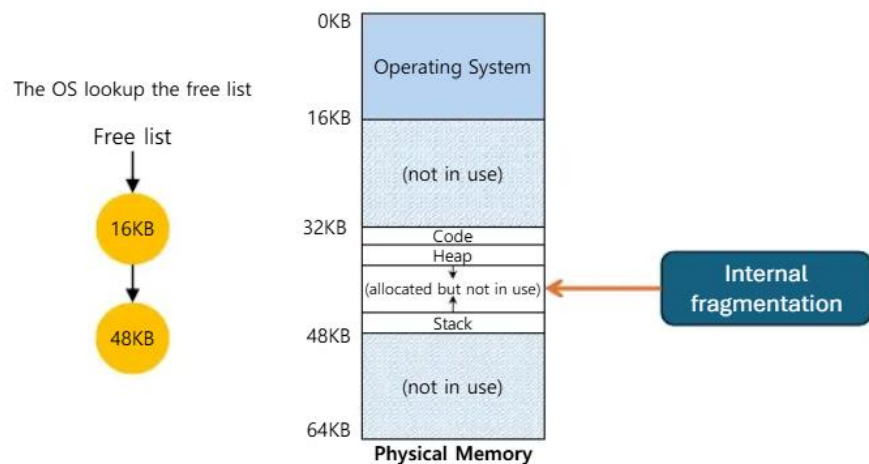
- Memory management
 - ♦ Manage memory via **free list** in general
 - Allocate memory for new processes, then remove the allocated memory space from the free list
 - Deallocate memory from terminated processes and insert the deallocated memory space back into free list
- Base/bounds management
 - ♦ Must set base and bounds registers properly upon context switch
- Exception handling
 - ♦ Exception handlers when exception arise
 - ♦ Likely action is to terminate offending process

OS Issues for Memory Virtualizing

- The OS must **take action** to implement **base-and-bounds** approach.
- Three critical junctures:
 - ◆ When a process **starts running**:
 - Finding space for address space in physical memory
 - ◆ When a process is **terminated**:
 - Reclaiming the memory for use
 - ◆ When context **switch occurs**:
 - Saving and storing the base-and-bounds pair

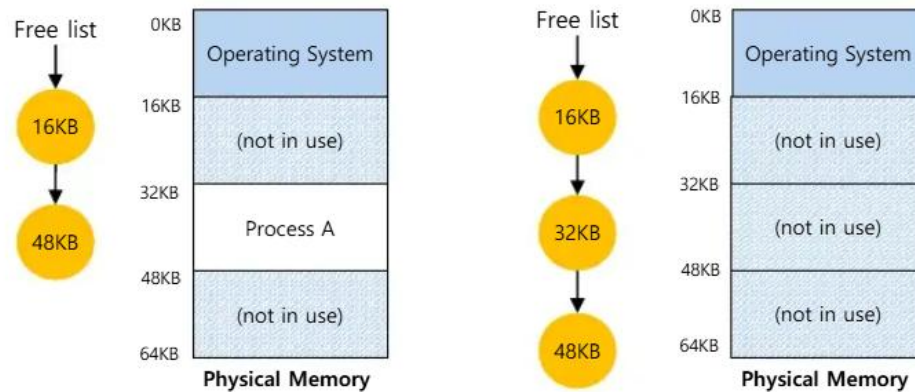
OS Issues: When a Process Starts Running

- When a process is created, the OS must **find space** for its address space in memory.
 - ◆ free list (data structure): A list of the range of the physical memory which are not in use.



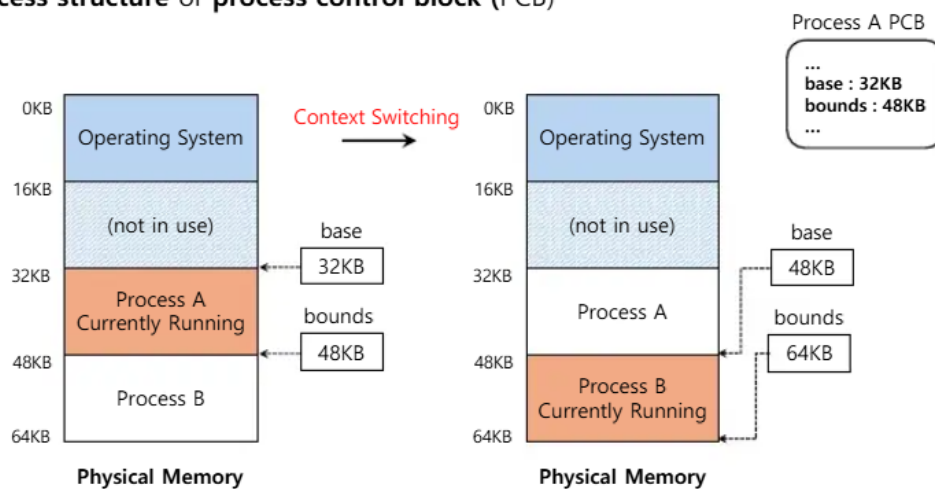
OS Issues: When a Process Is Terminated

- ▣ The OS must **put the memory back** on the free list.



OS Issues: When Context Switch Occurs

- ▣ The OS must **save and restore** the base-and-bounds pair.
 - ♦ In **process structure** or **process control block (PCB)**



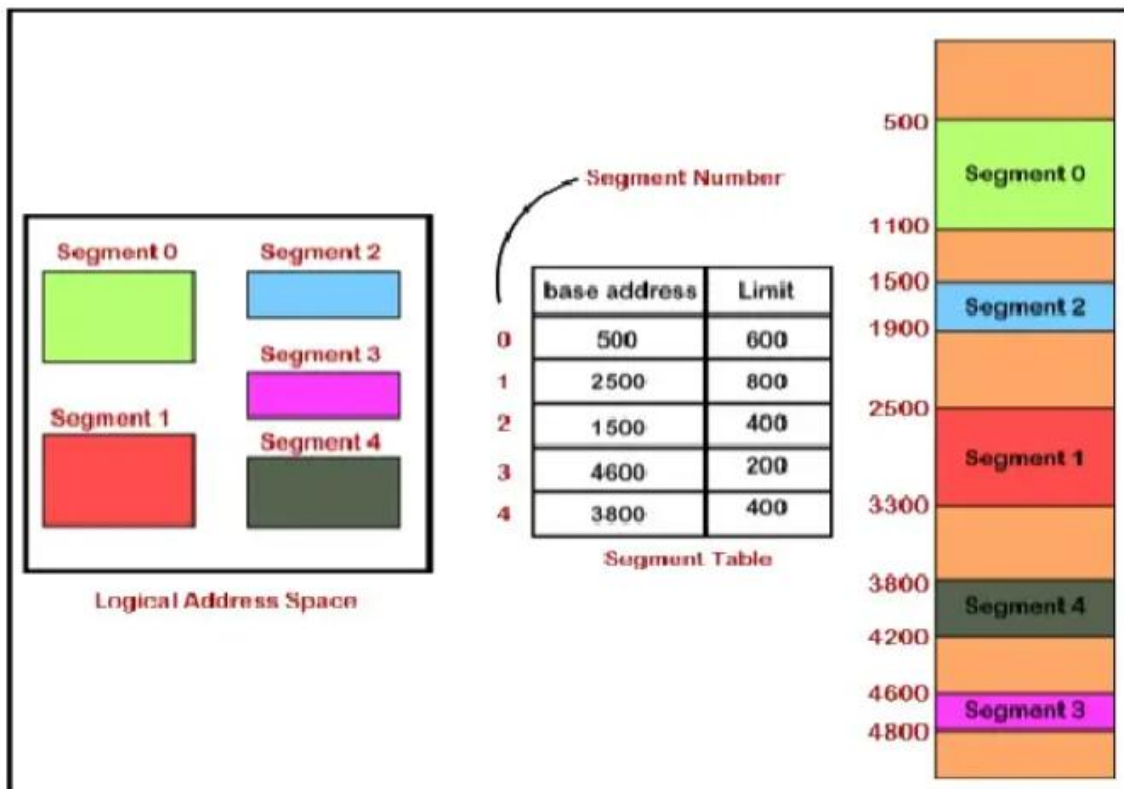
Segmentation

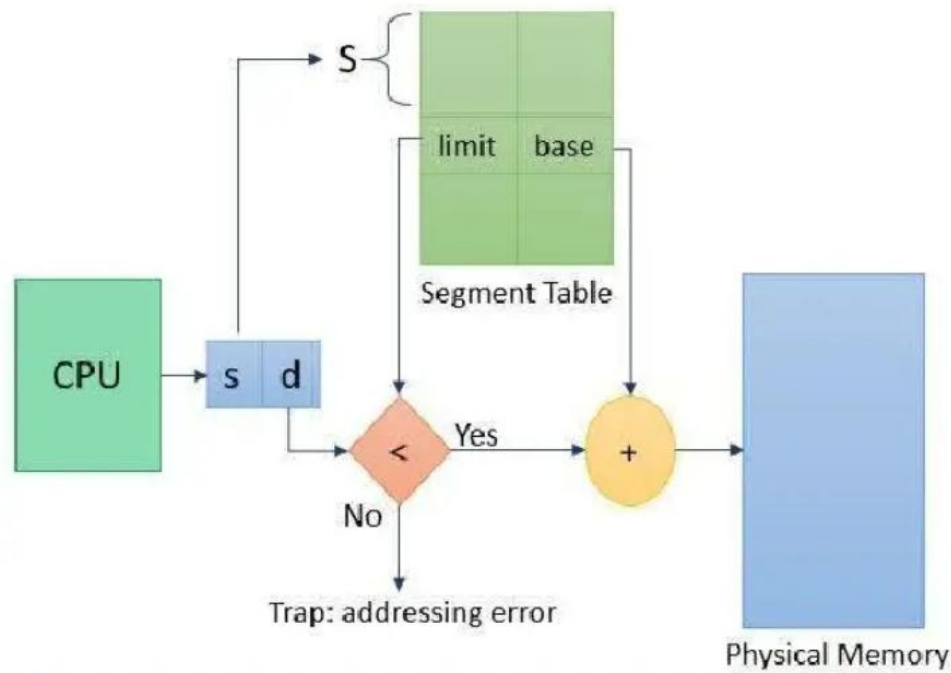
Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions. Each segment is actually a different logical address space of the program.

When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory.

Segmentation memory management works very similar to paging but here segments are of variable-length where as in paging pages are of fixed size.

A program segment contains the program's main function, utility functions, data structures, and so on. The operating system maintains a **segment map table** for every process and a list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory. For each segment, the table stores the starting address of the segment and the length of the segment. A reference to a memory location includes a value that identifies a segment and an offset.





Segment Table Segmentation

PAGING

* Non-Contiguous allocation (Virtual Memory).
→ wherever free space, OS can make use of free space

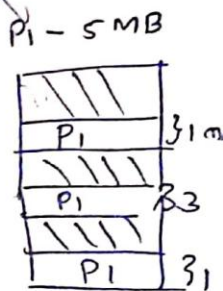
↳ To provide easy to the users. }
 ↳ To increase CPU utilization }

* OS giving illusion to the user such that

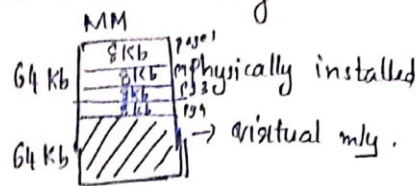
- User can write a very big program
- User thinks that its entire program is present in RAM.

- All the space allocated to the user is contiguous.

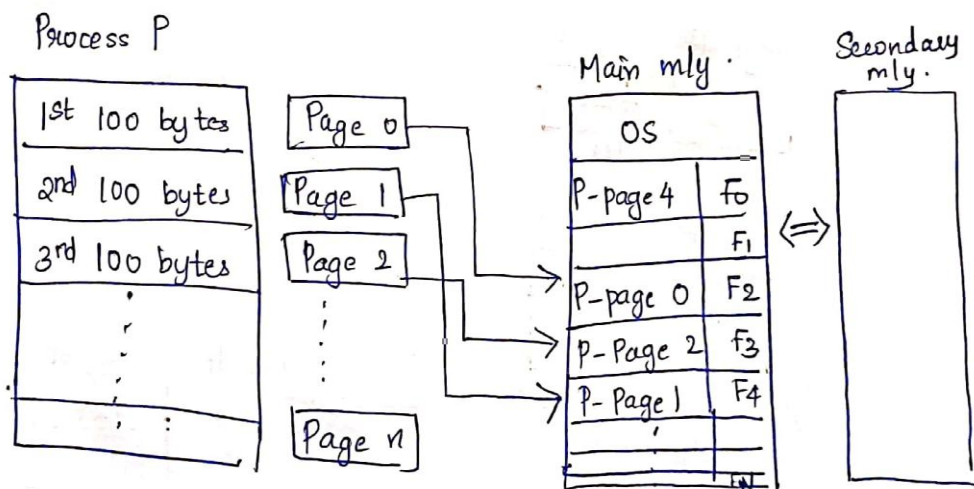
In reality, only a small portion of the user program is in RAM; which may or maynot be contiguous. while the remaining pgm is in secondary memory.



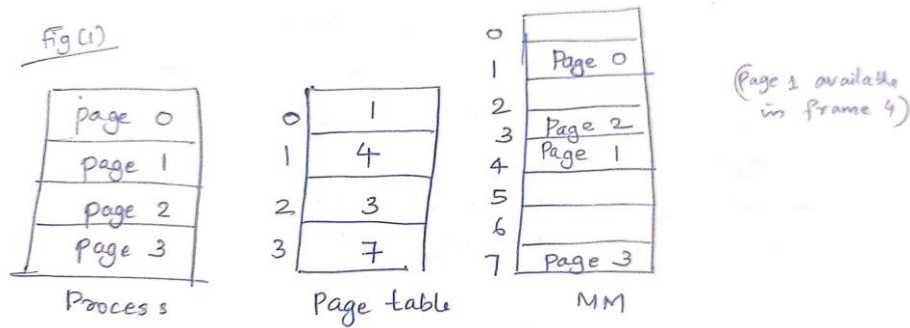
- * A computer can add more memory than the amount physically installed on system. This extra memory is called virtual memory.



- * Paging is a memory management technique in which process address space ^(virtual mly) is broken into blocks of same size called pages. Size of each page is same.
- * ~~Page~~ ^{Block} size $\rightarrow$ power of 2 (2, 4, 6, 8, ...)
- * The size of process can be measured in no. of pages. Similarly, main mly is divided into small fixed blocks (physically) of memory called frames. Size of each frame is same.
- * Size of frame = Size of page.
 - We can avoid External fragmentation.



- A single process is divided into fixed slots called Pages. These pages need to be stored in MM.
- MM is divided into frames & virtual mly into pages. Whatever data send from virtual mly to MM is in form of pages.
- ~~After~~ After execution, to make MM free, can swap data into secondary mly.

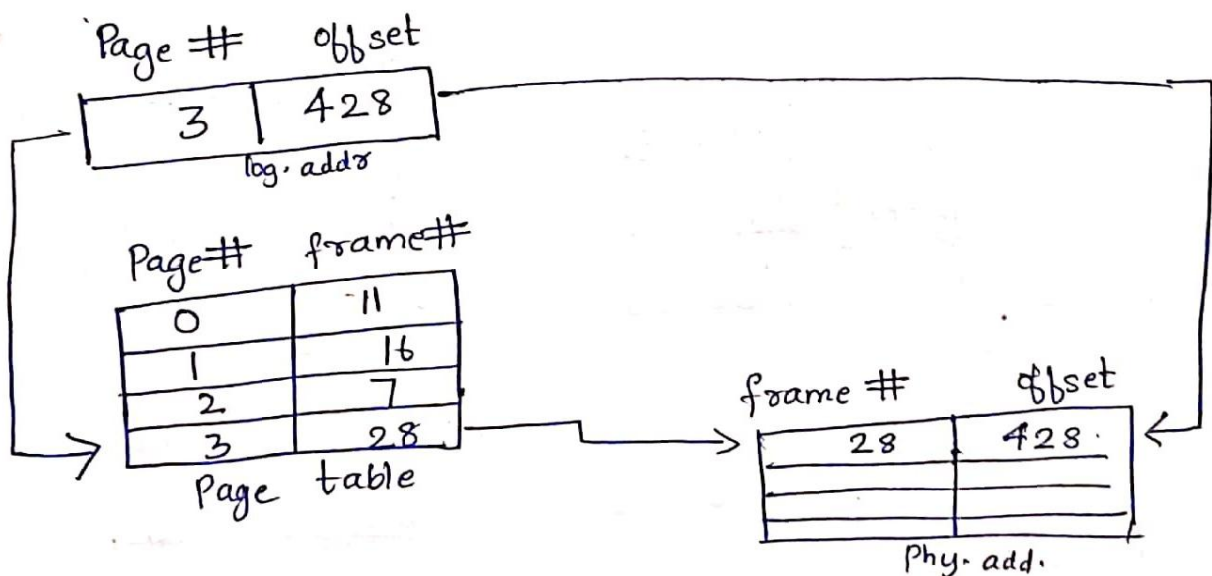
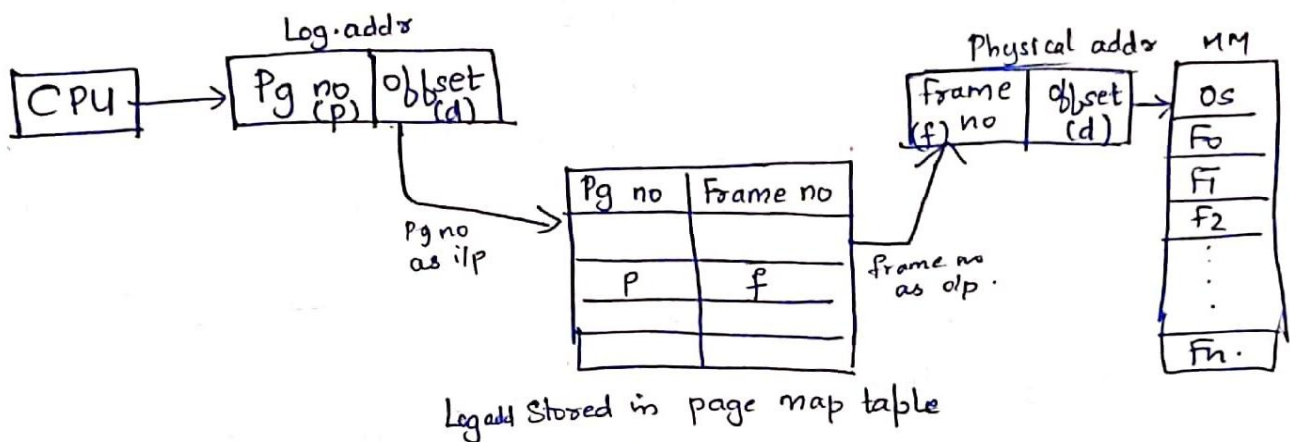


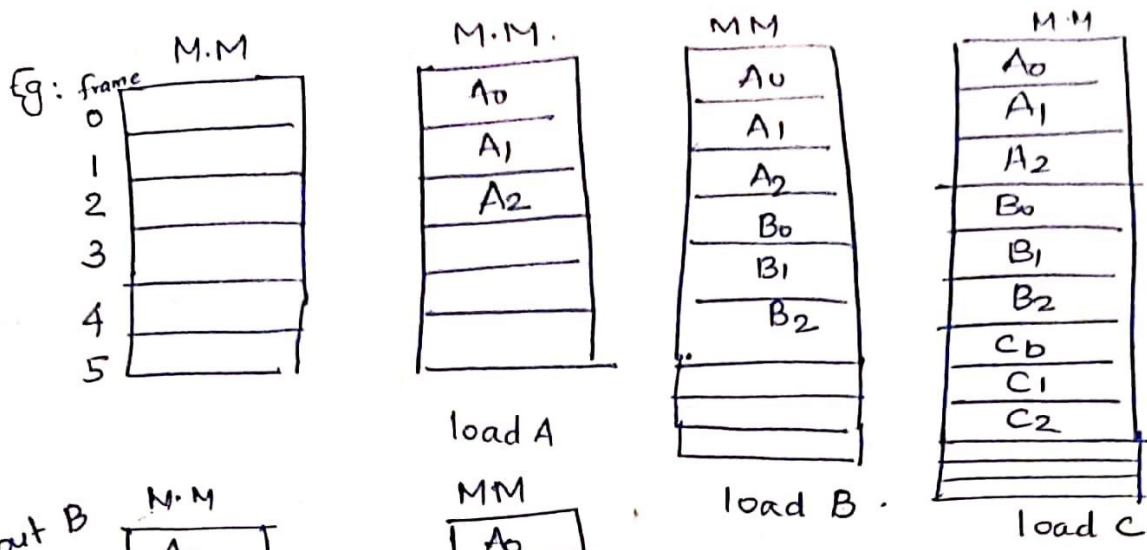
Address Translation (Paging Hardware)

- Page address is logical address & represented by,

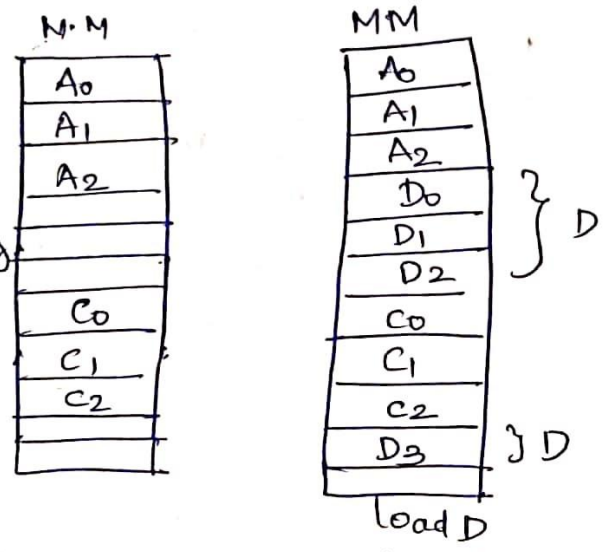
$$\text{logical addr} = \text{Pg no} + \text{page offset}$$
- Frame^{addresses} is called physical address & represented by,

$$\text{Physical addr} = \text{Frame no} + \text{Page offset}$$





Swap out B to disk, then the freed memory can be used by another process



Divided complete process D into 2 partitions.

Advantages & Disadvantages

1. Paging reduces external fragmentation but still suffers from internal fragmentation.
2. It is simple to implement.
3. Swapping becomes very easy because of ~~same~~ ^{equal} sizes of pages & frames.
4. It is not suitable for small RAM's.

Paging Example for a 32-byte memory with 4-byte pages

Process

Logical mly.

Page	0	a
0 →	1	b
	2	c
	3	d
1 →	4	e
	5	f
	6	g
	7	h
2 →	8	i
	9	j
	10	k
	11	l
3 →	12	m
	13	n
	14	o
	15	p

page table

0	5
1	6
2	1
3	2

Main mly or Physical mly

0		→ 0
4	i j k l	→ 1
8	m n o p	→ 2
12	.	→ 3
16		→ 4
20	a b c d	→ 5
24	e f g h	→ 6
28		→ 7

Size of page (4 bytes) = Size of frame (4 bytes)

Log add = 0 (a)

Page = 0

offset = 0

Phy. add = $Pr.no \times size + offset$
 $= 5 \times 4 + 0 = 20$

LA = 1

Pg = 0

offset = 1

Phy. add = $5 \times 4 + 1$
 $= 21 (b)$

LA = 4

Pg = 1

offset = 0

P.A = $6 \times 4 + 0$
 $= 24 (e)$

LA = 15

Pg = 3

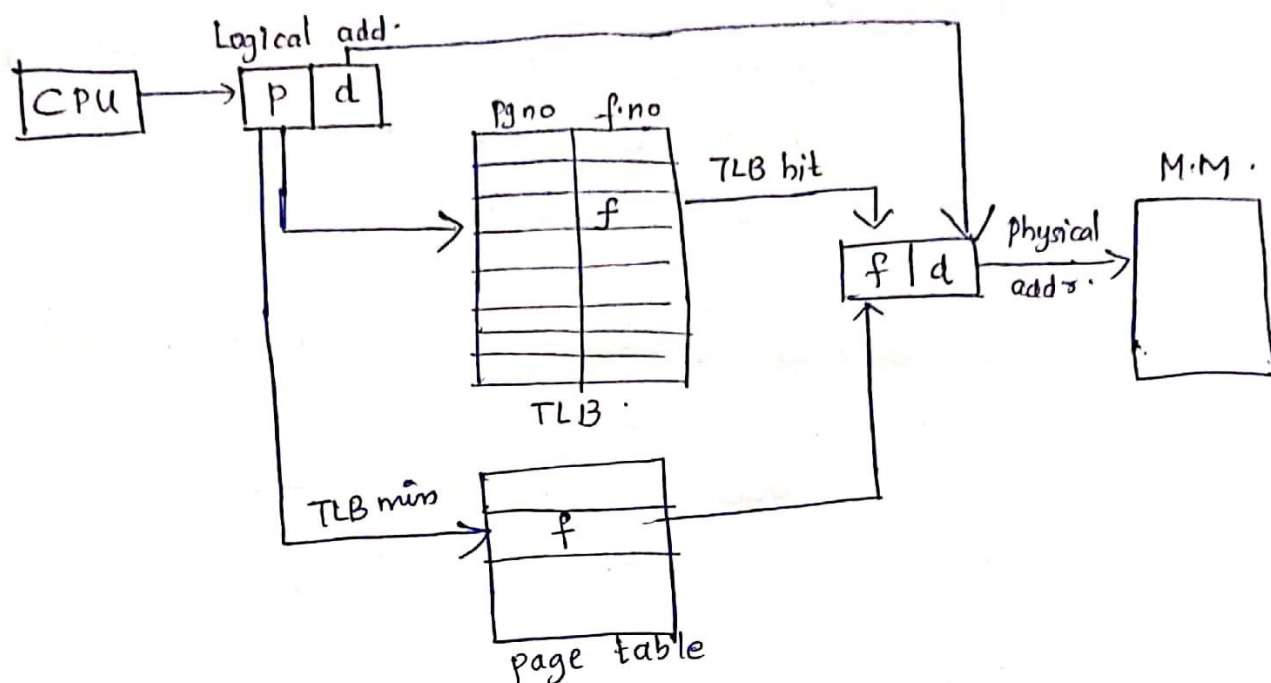
offset = 3

PA = $2 \times 4 + 3$
 $= 11 (p)$

Translation Look-aside Buffer (TLB)

10.

Paging Hardware with TLB



- * Page table is stored inside MM.
- * 2 memory access required:
 - Accessing page table
 - Accessing MM.
- * CPU generate LA (pg no, offset). Pg no given as i/p to page table. So in order to access pg table, require 1 mly access. ~~3~~
- * Frame no as o/p - Phy. add. (2nd mly. access).
- * ~~To overcome this~~ 2 mly access required for executing a single instn. To overcome this problem, TLB is used.
- * TLB contains the ^{most} frequently accessed ~~entry~~ ^{pages} in page table.
- * CPU generated LA [p, d]. Page no is compared with every page no. in TLB. ~~If m~~

- If match found, it is TLB hit & gives
frame no. as o/p.

& the frame no is combined with offset will
give physical address (1 mly access required)

- If match not found, it is TLB miss.

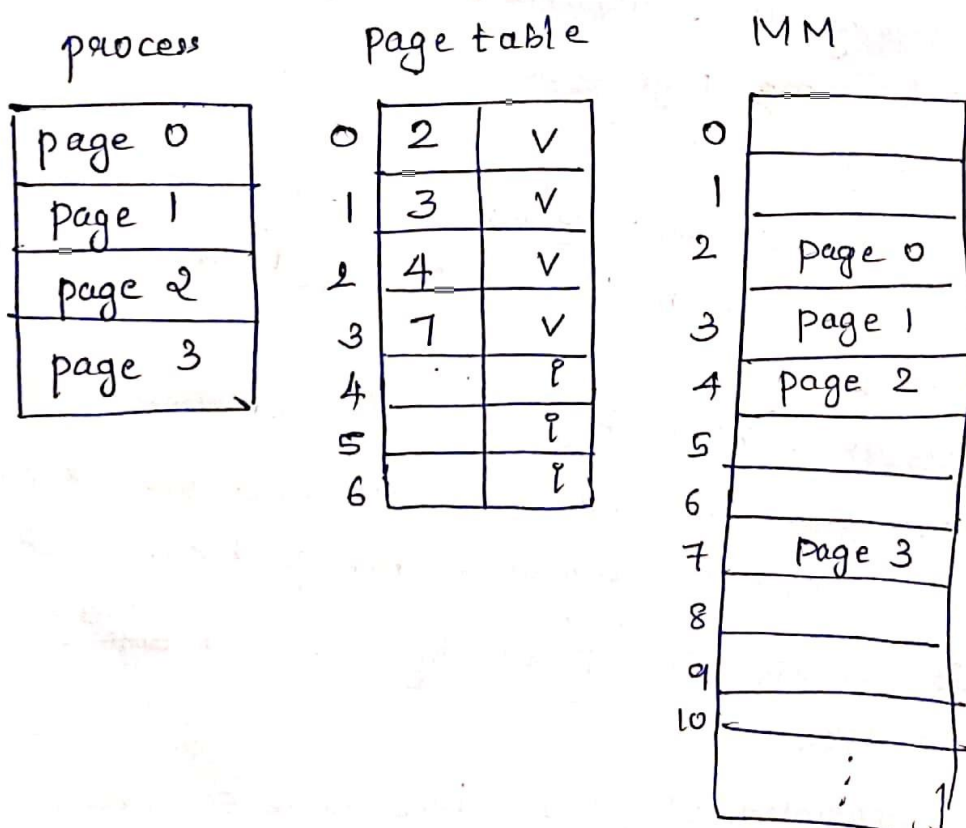
Page no given as i/p to page table. Produces frame
no as o/p & the frame no is combined with
offset will give physical address.

After getting frame no, the corresponding entry
ie, page no & frame no information will be
updated in TLB.

Suppose TLB is full, one of the row will be
replaced with this new entry.

Protection.

* ~~process~~ How to provide protection in paging.



- * Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed.
- * Valid-Invalid bit attached to each entry in the page table.
 - "Valid"^(v) indicates that the associated page is in the process's logical address space & is thus a legal page.
 - "Invalid"⁽ⁱ⁾ indicates that the page is not in the process's logical address space.

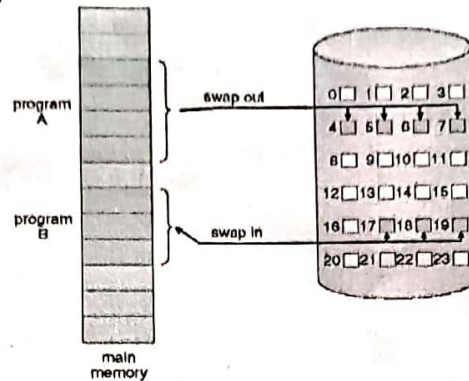
* Any violations, result in a trap to the kernel.

Shared Pages.

- * An advantage of paging is the possibility of sharing common code.
- * Shared code
 - One copy of read-only code shared among processes (ie, text editors)
 - Similar to multiple threads sharing the same process space.
- ~~* Private code & data.~~
- * Private code & data.
 - Each process keeps a separate copy of the code & data.
 - The pages for the private code & data can appear anywhere in the logical address space.

5.6 Demand paging

- Loading the entire program in physical memory at program execution time is not so good because the user doesn't need the entire program in memory at a time.
- An alternative strategy is to load pages only as they are needed. This technique is known as **demand paging**.
- With demand-paged virtual memory, pages are only loaded when they are demanded during program execution; pages that are never accessed are thus never loaded into physical memory.

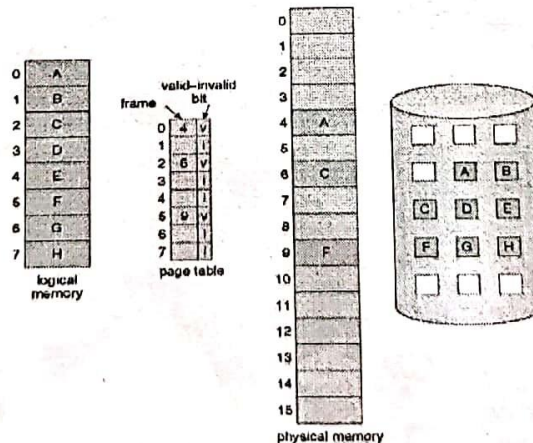


Transfer of a paged memory to contiguous disk space.

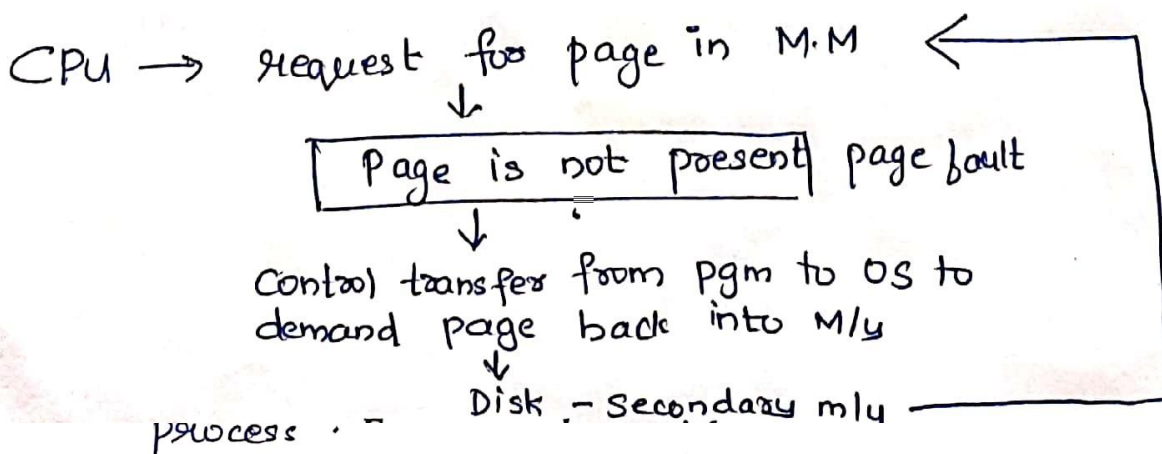
- A demand-paging system is similar to a paging system with swapping.
- But here it uses a lazy swapper; where it never swaps a page into memory unless that page will be needed.
- In Demand paging, **pager** is more suitable than swapper.

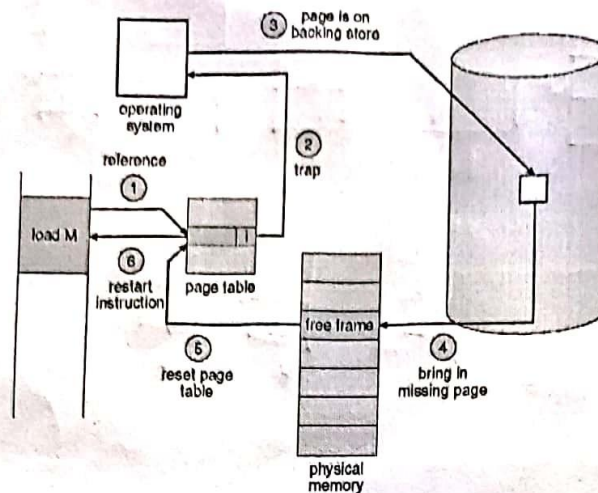
Basic Concepts

- With swapping, pager guesses which pages will be used before swapping out again
- Instead, pager brings in only those pages into memory
- some form of hardware support is needed to distinguish between the pages that are in memory and the pages that are on the disk.
- The valid-invalid bit scheme can be used for this purpose.
- With each page table entry a valid-invalid bit is associated ($v \Rightarrow$ in-memory – memory resident, $i \Rightarrow$ not-in-memory)
- Initially valid-invalid bit is set to i on all entries
- During MMU address translation, if valid-invalid bit in page table entry is i then a page fault occurs.



Page table when some pages are not in main memory





Steps in handling a page fault.

- If there is a reference to a page, first reference to that page will trap to operating system:
 - page fault
- The procedure for handling this page fault is
 1. Check an internal table (usually kept with the process control block) for this process to determine whether the reference was a valid or an invalid memory access.
 2. If the reference was invalid, terminate the process. If it was valid, but have not yet brought in that page, page it in.
 3. Find a free frame.
 4. Schedule a disk operation to read the desired page into the newly allocated frame.
 5. When the disk read is complete, modify the internal table kept with the process and the page table to indicate that the page is now in memory.
 6. Restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.
- In the extreme case, we can start executing a process with no pages in memory. When the operating system sets the instruction pointer to the first 404 Chapter 9 Virtual Memory instruction of the process, which is on a non-memory-resident page, the process immediately faults for the page. After this page is brought into memory, the process continues to execute, faulting as necessary until every page that it needs is in memory. At that point, it can execute with no more faults. This scheme is **pure demand paging**: never bring a page into memory until it is required
- Theoretically, some programs could access several new pages of memory with each instruction execution, possibly causing multiple page faults per instruction. This situation would result in unacceptable system performance. Locality of reference can be used for such situations.
- Hardware support needed for demand paging
 - **Page table.** This table has the ability to mark an entry invalid through a valid-invalid bit or a special value of protection bits.
 - **Secondary memory.** This memory holds those pages that are not present in main memory. The secondary memory is usually a high-speed disk. It is known as the swap device, and the section of disk used for this purpose is known as swap space

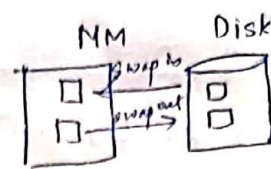
Performance of Demand Paging

- Demand paging can significantly affect the performance of a computer system.
- the effective access time is

Effective access time = $(1 - p) \times m_a + p \times \text{page fault time}$.

- where,
 - p be the probability of a page fault ($0 \leq p \leq 1$).
 - m_a be the memory-access time

If $p=0$, no page fault
 $p=1$, every reference is a fault.

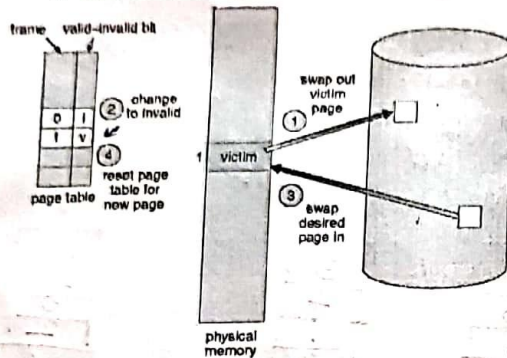


Page Replacement

- It prevents over-allocation of memory by modifying page-fault service routine to include page replacement. ** Use modify (dirty) bit to reduce overhead of page transfers.*

Basic Page Replacement

- Page replacement takes the following approach.
 1. Find the location of the desired page on the disk.
 2. Find a free frame:
 - a. If there is a free frame, use it.
 - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
 - c. Write the victim frame to the disk; change the page and frame tables accordingly.
 3. Read the desired page into the newly freed frame; change the page and frame tables.
 4. Restart the user process.

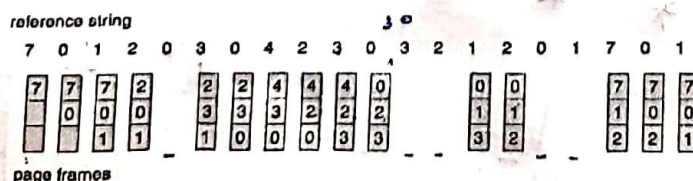


page replacement

- Use **modify (dirty) bit** to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory.
- two major problems to implement demand paging
 - Frame allocation algorithm
 - page replacement algorithm
- **Frame-allocation algorithm** determines
 - How many frames to give each process
 - Which frames to replace
- **Page-replacement algorithm**
 - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
 - String is just page numbers, not full addresses
 - Repeated access to the same page does not cause a page fault
 - Results depend on number of frames available

First-In-First-Out (FIFO) Algorithm

- A FIFO replacement algorithm associates with each page the time when that page was brought into memory.
- When a page must be replaced, the oldest page is chosen.
- Example:
 - Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
 - 3 frames (3 pages can be in memory at a time per process)



- **Belady's Anomaly** - for some page-replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.

$$\text{Page Hit} = 7$$

$$\text{Page fault} = 15$$

$$\text{Hit ratio} = \frac{\text{no. of hit}}{\text{Total no. of reference}} = \frac{7}{22} \times 100$$

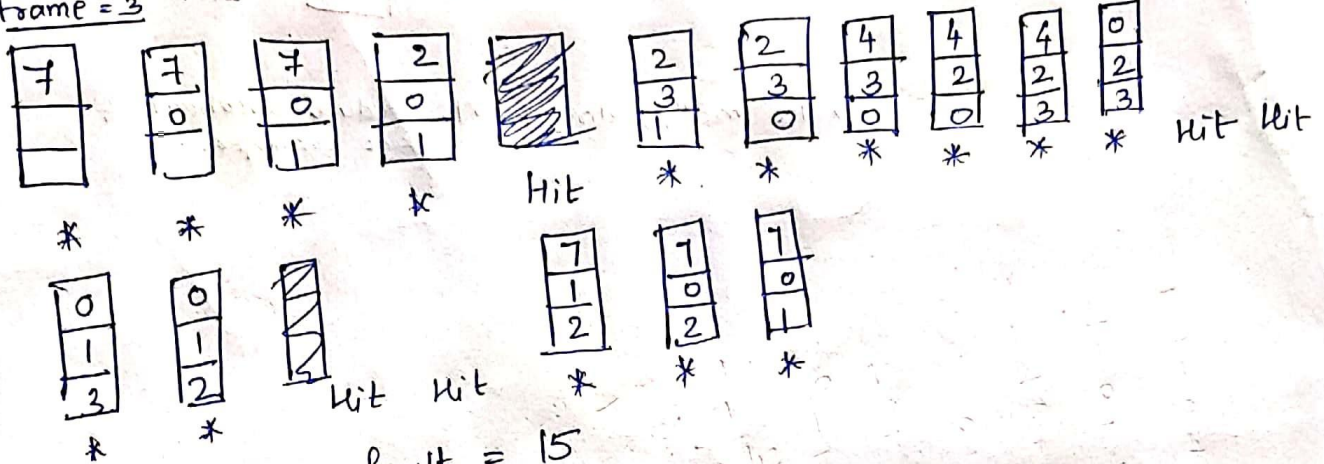
$$\text{Page fault ratio} = \frac{\text{no. of Page fault}}{\text{Total no. of ref}} = \frac{15}{22} \times 100$$

First In First Out (FIFO)

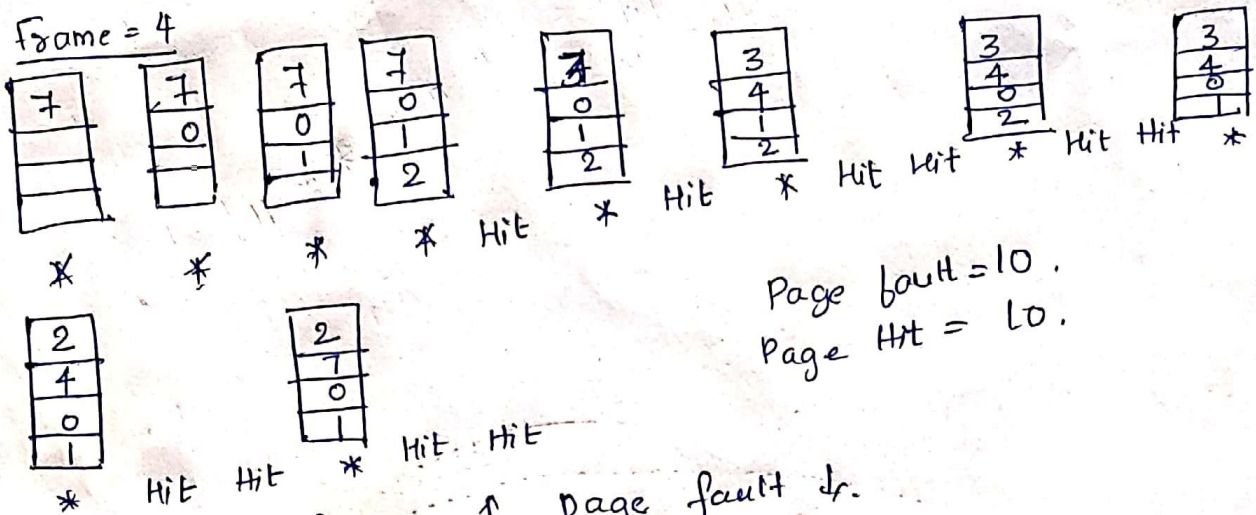
16

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Frame = 3



Frame = 4

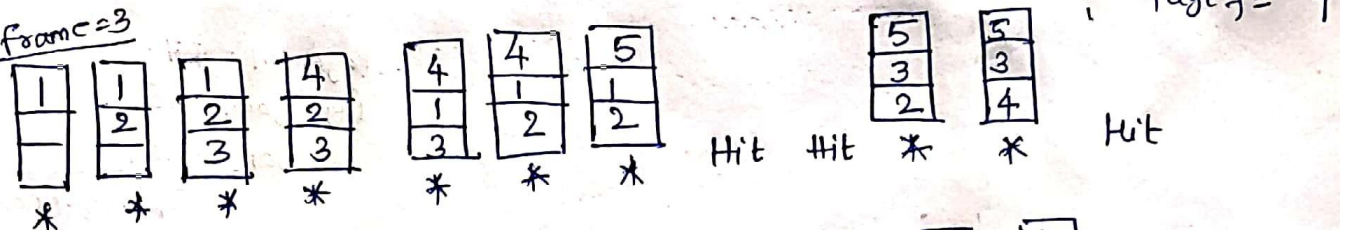


As no. of frames ↑, page fault ↓.

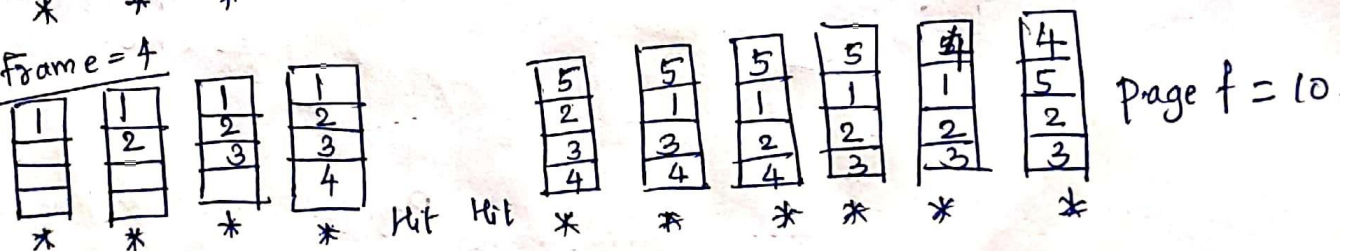
Belady Anomaly in FIFO.

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Frame = 3



Frame = 4



As no. of frames ↑, page fault also ↑. This unexpected condition is called belady's ~~Anom~~ Anomaly.

OPT $\rightarrow$ 2 3 2 1 5 2 4 5 3 2 5 2 P.f = 6.

18¹²

2	2	2	2	4	2	2						
3	3	3	3	3	3	3						
		1	5	5								
*	*	H	*	*	H	*	H	*	H	*	H	*

Hit = 6
Hit $\sigma = 6/12$
Miss $\sigma = 6/12$

- Example: For the reference string 1,2,3,4,1,2,5,1,2,3,4,5
page fault is 9 with frames 3 and page fault is 10 with frames 4.

Optimal Page Replacement

- Replace page that will not be used for longest period of time
- It has the lowest page-fault rate of all algorithms and will never suffer from Belady's anomaly-hence known as OPT or MIN.

P.f = 9, H.it = 11

Hit ratio = 11/20

Miss ratio = 9/20

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2		2		2		2		7				
*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*

page frames

- It is difficult to implement, because it requires future knowledge of the reference string.

LRU Page Replacement (Least Recently Used Alg (LRU))

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page

P.f = 12

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1			
*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*

page frames

- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- Two methods of implementations:
 - Counter implementation
 - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
 - When a page needs to be changed, look at the counters to find smallest value (less frequently used)
 - Search through table needed
 - Stack implementation
 - Keep a stack of page numbers in a double link form:
 - Page referenced:
 - move it to the top
 - requires 6 pointers to be changed
 - But each update more expensive
 - No search for replacement
- LRU and OPT are cases of stack algorithms that don't have Belady's Anomaly

LRU $\rightarrow$ 2 3 2 1 5 2 4 5 3 2 5 2 P.f = 7

2	2	2	2		2		3		3			
*	*	H	*	*	H	*	H	*	*	H	H	*

FIFO

Pages:- 3 2 1 3 4 1 6 2 4 3 4 2 1 4 5 2 1 3 4

	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
F ₁	3	3	3		4		4	4		3	3	3		5	5		5	5	4
F ₂		2	2		2		6	6		6	4	4		4	2		2	2	
F ₃			1		1		1	2		2	2	2	2	1	1		3	3	4
	*	*	*	H	*	H	*	*	H	*	*	*	*	H	*	*	H	*	*

No. of page faults = 13.

Hit ratio = $\frac{\text{No. of hit}}{\text{Total}}$

$$= \frac{6}{19}$$

Miss ratio = $\frac{\text{No. of miss}}{\text{Total}}$

$$= \frac{13}{19}$$

OPT

Replace a page which is not in used ^{in near future} for a long time.

	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
F ₁	3	3	3		4		4			4			4		5			5	5
F ₂		2	2		2		2			2			2		2			3	3
F ₃			1		1		6			3			1		1			1	4
	*	*	*	H	*	H	*	H	H	*	H	H	*	H	*	H	H	*	*

No. of page fault = 10.

Hit = 9

Hit ratio = $\frac{9}{19}$

LRU

Replace a page which is not in use for a long time

	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
F ₁	3	3	3		3		6	6	6	3			1		1	2	2	2	4
F ₂		2	2		4		4	2	2	2			2		5	5	5	3	3
F ₃			1		1		1	1	4	4			4		4	4	1	1	1
	*	*	*	H	*	H	*	*	*	*	H	H	*	H	*	*	*	*	*

Page fault = 14.

hit ratio = $\frac{5}{19}$

Compare FIFO, OPT, LRU.

19

- * Whenever CPU needs to execute a page, it checks whether that page is present in MM or not.
If page present in MM, then CPU executes that page.
If page not present in MM, then it has page fault.
Whenever a page fault occurs, OS loads page from Hard disk to MM.

- * If MM is full, one page needs to be replaced. That is decided with page replacement algorithm.

FIFO - Page that has been in mly for the longest period of time will be replaced i.e., oldest page-replaced.

OPT - Page that will not be used for longest period of time in the future will be replaced.

LRU - Page that is least recently used in the past will be replaced.

- * OPT is the best alg. bcoz OPT produces less no. of page faults than FIFO & LRU. But it is impossible to implement OPT alg practically bcoz OPT needs to know future page information also.

- * LRU is best compared to FIFO as it produces less page fault than FIFO.

- * In increasing frame size, no. of page faults must be less.

- * In FIFO, it has belady's anomaly. (i.e., even though by increasing the no. of frames, the no. of page faults was also more)

- **Page fault and swapping:** We know every program divided into some pages. When a program need a page which is not in RAM that is called page fault.
- Whenever a page fault happens, operating system will try to fetch that page from secondary memory and try to swap it with one of the page in RAM. This is called swapping.

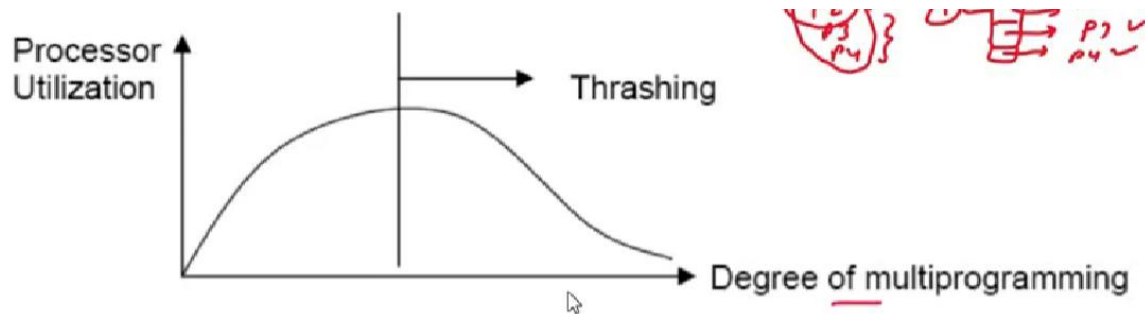
m (OS) Thrashing & Prevention Thrashing is a pathological state where a computer's virtual memory subsystem is in a constant state of paging (swapping data between RAM and disk), resulting in extremely high disk activity and low CPU utilization.

Causes of Thrashing:

- **High Degree of Multiprogramming:** Too many processes running simultaneously.
- **Insufficient Physical Memory (RAM):** Not enough RAM to hold the active "working sets" of all running processes.
- **Poor Locality:** Processes jumping between memory locations, causing frequent page faults.

Methods to Prevent/Solve Thrashing:

1. **Increase RAM:** The most effective, long-term solution to accommodate more processes.
2. **Reduce Degree of Multiprogramming:** Terminate or suspend active processes to free up frames.
3. **Working Set Model:** The OS monitors the pages a process is actively using (its "working set"). If the sum of working sets exceeds physical memory, the OS suspends processes to prevent thrashing.
4. **Page-Fault Frequency (PFF):** Monitor the rate of page faults. If a process exceeds a certain upper limit, give it more frames. If it falls below a lower limit, take frames away.
5. **Local Page Replacement:** Restrict processes to using their own allocated frames rather than stealing from others, which prevents one thrashing process from taking down the whole system.



- To prevent thrashing, we must provide a process as many frames as it needs. For this, a model called *the Working Set Model* is developed which depends on the **locality model** of program

Working Set Model (Simple Notes)

◆ Definition

The **Working Set** of a process is the set of pages that the process is currently using in memory.

It helps to **prevent thrashing**.

◆ What is Working Set (WSS_i)?

- It is the set of pages referenced in the last **Δ (delta) time**.
- Δ = fixed time window (recent references).

$$WSS_i = \text{Pages used in last } \Delta$$

◆ Total Memory Demand

$$D = \sum WSS_i$$

- D = Total frames needed by all processes
- M = Total frames available in memory

◆ Condition

- If **D ≤ M** →  No Thrashing

- If $D > M \rightarrow \text{X}$ Thrashing occurs
-

◆ How It Prevents Thrashing

1. OS calculates working set of each process.
2. Adds total demand (D).
3. If memory is not enough:
 - Suspend some processes.
 - Reduce degree of multiprogramming.

Page Fault Frequency (PFF)

◆ Definition

Page Fault Frequency (PFF) is a technique used to **control thrashing** by monitoring the page fault rate of a process.

◆ Basic Idea

- If a process has **too many page faults** $\rightarrow$ it needs more frames.
- If a process has **very few page faults** $\rightarrow$ it has extra frames.

The OS adjusts frames based on page fault rate.

◆ How It Works

1. OS sets:
 - ▲ Upper limit (maximum page fault rate)
 - ▼ Lower limit (minimum page fault rate)
2. Compare page fault rate:
 - If rate $>$ **Upper limit**
 $\rightarrow$ Give more frames to the process.
 - If rate $<$ **Lower limit**
 $\rightarrow$ Remove some frames from the process.

Page Fault Frequency

- ◆ Direct approach to handle thrashing that controls page fault rate

